

C Programming

7_functions_p2

functions

```
returnType functionName(arguments) {  
    // function body  
    return (value);  
}
```

```
типВозвращаемогоЗначения имяФункции(Аргументы)  
{  
    // Тело функции  
    return (Возвращаемое значение);  
}
```

Recursion (рекурсия)

Рекурсия — это поведение функции, при котором она вызывает сама себя.

```
int factorial_non_recursive(int N) {  
    int res = 1;  
    for (int i = N; i > 1; i--) {  
        res = res * i;  
    }  
    return res;  
}  
// ..  
int n = factorial_non_recursive(5);
```

```
int factorial(int N) {  
    if (N < 1) return 1;  
    return N * factorial(N - 1);  
}  
// ..  
int n = factorial(5);
```

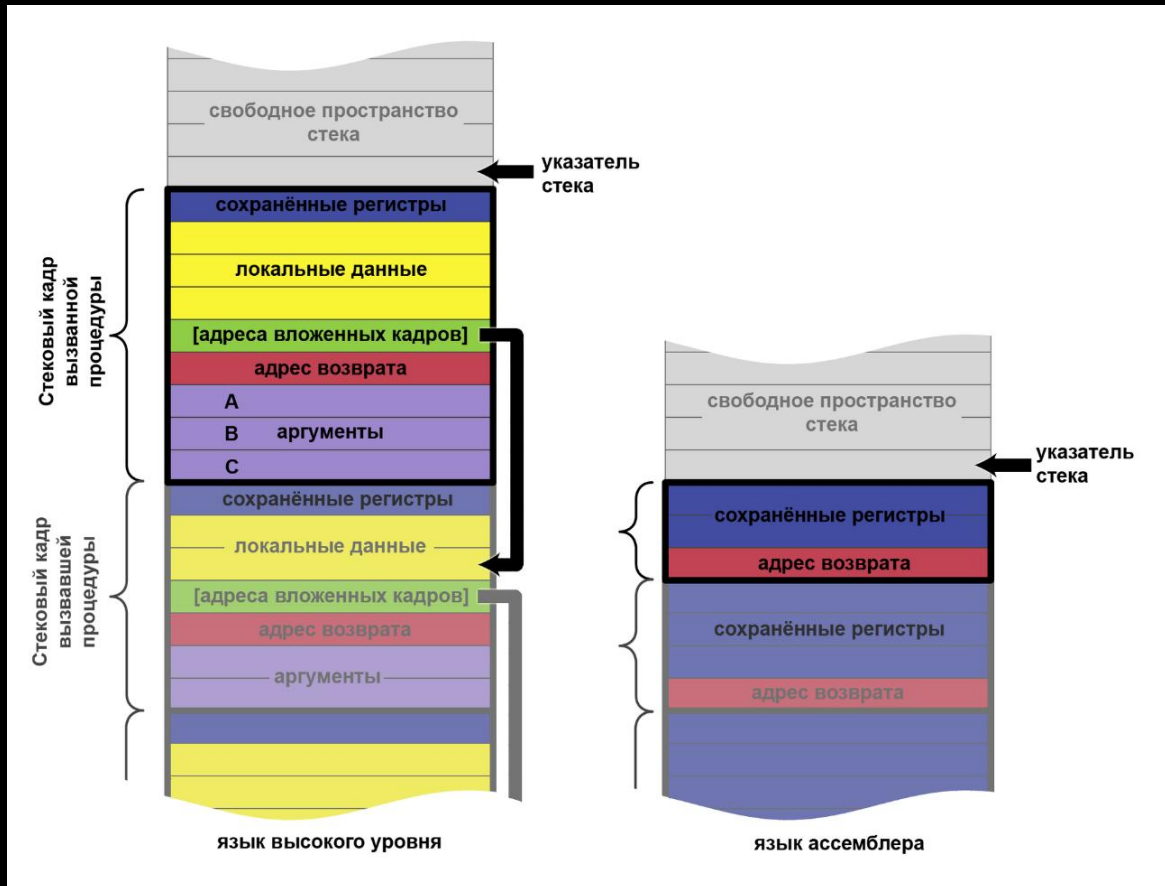
```

/*
 * Раскрытие вычислений:
 *
 * factorial(5) = 5 * factorial(4)
 *              = 5 * (4 * factorial(3))
 *              = 5 * (4 * (3 * factorial(2)))
 *              = 5 * (4 * (3 * (2 * factorial(1))))
 *              = 5 * (4 * (3 * (2 * (1 * factorial(0)))))
 *              = 5 * (4 * (3 * (2 * (1 * 1))))
 *              = 5 * (4 * (3 * (2 * 1)))
 *              = 5 * (4 * (3 * 2))
 *              = 5 * (4 * 6)
 *              = 5 * 24
 *              = 120
 */

int factorial(int N) {
    if (N < 1) return 1;
    return N * factorial(N - 1);
}
// ..
int n = factorial(5);

```

Function frame



Кадр функции состоит из локальных данных, входных аргументов в функцию, а также, некоторых важных адресов - один из них адрес возврата, который необходим примерно для того, чтобы после вызова функции мы могли вернуться в место откуда эту ф-ию вызывали, продолжив работу.

- Во-первых, локальные переменные и аргументы они существуют только пока существует функция.
- Во-вторых, вызывая ф-ию мы создаем кадр этой функции над кадром другой функции, занимая в памяти место, что если у нас будет ф-ия бесконечно рекурсировать?
- Ну и например, что у ф-ий есть какие-то адреса?..

https://ru.wikipedia.org/wiki/Стековый_кадр

```
void swap(int a, int b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}  
// ..  
int a = 5;  
int b = 37;  
swap(a, b);  
// a = ? b = ?
```

При вызове ф-ии мы передаем в нее в качестве аргументов **значения** переменных, а не сами переменные, т.е. при создании кадра ф-ии `swar()` создаются локальные копии переменных `a` и `b`.

Получается, что `a` и `b` в `swar()` и `a` и `b` до вызова `swar` - локальные переменные своих функций

```
void swar(int a, int b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}
```

```
void swap(int *a, int *b) {  
    int tmp = *a;  
    *a = *b;  
    *b = tmp;  
}  
// ..  
int a = 5;  
int b = 37;  
swap(&a, &b);
```

Return via pointer

```
void add(int a, int b, int *res) {  
    *res = a + b;  
}  
// ..  
int res;  
add(5, 3, &res); // res = 8
```

- Таким же образом можно из ф-ий возвращать больше одного элемента, т. к. return умеет возвращать лишь одно значение
- Массивы не копируются в функцию, а передаются по адресу

```
void swap(int *ptr1, int *ptr2)
{
    printf("swap: %d\n", *ptr1); // 5
    ptr1 = ptr2;
    printf("swap: %d\n", *ptr1); // 37
}
// ..
int a = 5;           // &a = 1000
int b = 37;          // &b = 2000
int *ptr = &a;       // &ptr = 3000

printf("%d\n", *ptr); // 5
swap(ptr, &b);
printf("a = %d\n", a); // 5
printf("%d\n", *ptr);  // 5
```

```

void swap(int *ptr1, int *ptr2)
{
    printf("swap: %d\n", *ptr1); // 5
    ptr1 = ptr2;
    printf("swap: %d\n", *ptr1); // 37
}
// ..
int a = 5;           // &a = 1000
int b = 37;          // &b = 2000
int *ptr = &a;       // &ptr = 3000

printf("%d\n", *ptr); // 5
swap(ptr, &b);
printf("a = %d\n", a); // 5
printf("%d\n", *ptr);  // 5

```

a		b		<u>ptr</u>		ptr1		ptr2
5		37		1000		1000		2000
1000		2000		3000		4000		5000

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
```

```
    for (int i = 0; i < argc; i++) {
```

```
        printf("argv[%d] = %s\n", i, argv[i]);
```

```
    }
```

```
}
```

- argc - целое число, количество аргументов, всегда как минимум будет один нулевой аргумент - название исполняемого файла
- argv - массив строк, в нем будут аргументы в виде строк, индексы строк от 0 до argc - 1 включительно

cowsay its the first day of the month

cowsay -f dragon rawr

Dangling pointer (Висячий указатель)

```
char *f() {  
    char s  
    return &s;  
}
```

Functions pointers (указатель на функцию)

```
int (*cipher_func)(int, char *);  
  
cipher_func = snow3g;  
  
cipher_func();
```

```
int snow3g(int key, char *text) {  
    // ..  
}  
  
int aes(int key, char *text) {  
    // ..  
}  
  
int grasshopper(int key, char *text) {  
    // ..  
}
```

`cipher_func` – указатель на функцию * с аргументами `int`, `char *`, возвращающей тип данных `int`

Function pointer array

```
void (*dfts_fun[])(float, int) = { dft128, dft256, dft512, dft1024 };  
  
dfts_fun[1](/*аргументы*/); // вызовется ф-ия dft256()
```

dfts_fun — это массив указателей на функцию, принимающей аргументы типа float и int и возвращающей значение типа float

cdecl

C gibberish ↔ English

```
char ((*x())[5])()
```

declare x as function returning pointer to array 5 of pointer to function returning char

[permalink](#)

<https://cdecl.org/>

Variadic functions (Вариативные ф-ии)

```
#include <stdarg.h>

int my_log_print(char *format, ...) {
    va_list args;

    va_start(args, format);

    // ...

    va_end(args);

    return 0;
}
```

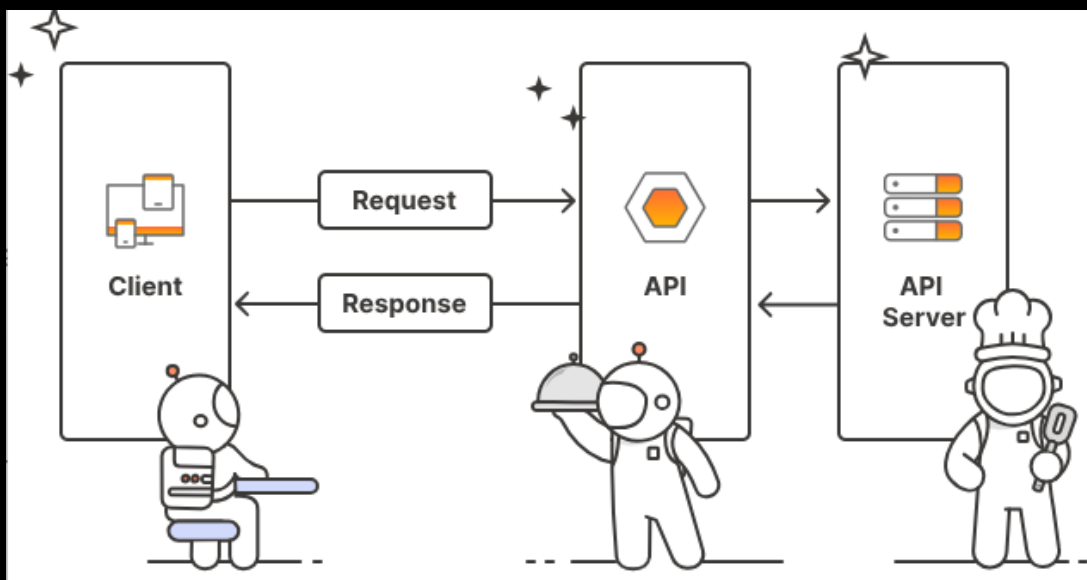
Callback

Callback (англ. call — вызов, англ. back — обратный) или функция обратного вызова в программировании — передача исполняемого кода в качестве одного из параметров другому коду.

Простыми словами это функция, которая передаётся в другую функцию как аргумент и вызывается позже (когда произойдёт какое-то событие или завершится операция)

API

API (Application Programming Interface) – программный интерфейс приложения, описывает протокол взаимодействия между программами. Представляет собой набор классов, функций, структур или констант.



Сайты/приложения:

- Погоды
- Банковские
- Карты (2 Gis, Google maps, Yandex maps)
- Боты в мессенджерах (VK, Telegram)

Простыми словами:

Это набор правил и инструкций, который позволяет одним программам общаться с другими.

API говорит: «Если ты хочешь получить от меня какие-то данные или выполнить действие, обратись ко мне вот таким способом, и я тебе вот так отвечу».

ABI

ABI (Application Binary Interface) – двоичный интерфейс приложения, описывает набор соглашений для доступа приложения к операционной системе и низкоуровневым сервисам.

- Как передаются аргументы в функцию? (В регистры процессора или в стек? В каком порядке?)
- Как функция возвращает результат?
- Как данные располагаются в памяти?
- Какой формат исполняемого файла? (ELF в Linux)
- Как происходит системный вызов (обращение к ядру)

Простыми словами:

это низкоуровневое соглашение о том, как одна скомпилированная программа (уже в виде нулей и единиц) должна взаимодействовать с другой на уровне процессора и операционной системы.

API - общение программ между собой, ABI - общение программ с машиной

Github

<https://github.com/kruffka/C-Programming>

